4 - 回答模块和 AI 集成

本节重点

学习 OpenAI API 的集成和调用技巧，掌握 AI 大模型的参数调优方法，实现智能回答功能，并为系统提供强大的 AI 能力支撑。

具体内容包括：

- OpenAI API 分析
- OpenAI API 集成
- 回答者模块

💡 这一节也是项目的核心，尤其是 AI 大模型的调用技巧在很多 AI 项目中都会用到。不过由于 AI 大模型更新飞速，可能有些调用参数已经发生了变化，大家重点还是学习思路和方法，实际请求细节以官方文档为准。

一、OpenAI API 分析

OpenAI 平台介绍

OpenAI 是全球领先的人工智能研究公司，提供了包括 GPT-5、DALL-E 等在内的多个强大的 AI 模型。

对于开发者来说，OpenAI 提供了简单易用的 REST API，让我们可以轻松地在应用中集成 AI 能力。

现在很多大模型的接口和 SDK 都是兼容 OpenAI 规范的，只要学会了 OpenAI 的 API，其他大模型的调用都是洒洒水~

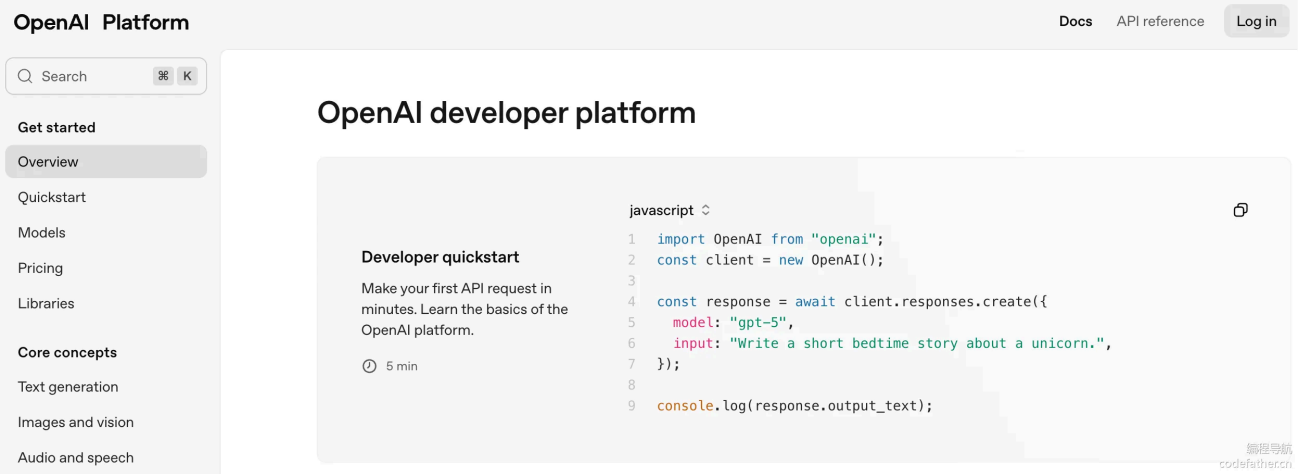
使用 OpenAI 等大模型时需要考虑一些因素：

- 成本：按 token 计费，大量使用成本较高
- 网络：需要稳定的网络环境访问（尤其是国外服务）
- 限制：有请求频率和内容审核的限制

OpenAI API 详解

1、API Key 获取

首先需要在 OpenAI 官网 注册账号并获取 API Key：



注意，API Key 相当于你的身份凭证，不要泄露给他人！也不要将 API Key 硬编码在代码中或提交到公开仓库！建议通过环境变量或配置文件管理 API Key。

2、Completions API

项目中使用的是 OpenAI 的 Completions API，这是一个通用的文本生成接口：

请求地址：`https://api.openai.com/v1/completions`

请求方法：POST

请求头：

▼
plain
复制代码
```
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json
```

请求参数：

▼
json
复制代码
```
{
    "model": "text-davinci-003",
    "prompt": "你好，我是程序员鱼皮",
    "max_tokens": 1024,
    "temperature": 0.7,
    "top_p": 1,
    "frequency_penalty": 0,
    "presence_penalty": 0
}
```

响应格式：

▼
json
复制代码
```
{
    "id": "cmpl-xxx",
    "object": "text_completion",
    "created": 1677652288,
    "model": "text-davinci-003",
    "choices": [
        {
            "text": "你好！很高兴认识你...",
            "index": 0,
            "logprobs": null,
            "finish_reason": "stop"
        }
    ],
    "usage": {
```

```
        "prompt_tokens": 10,
        "completion_tokens": 50,
        "total_tokens": 60
    }
}
```

3、关键参数详解

理解这些参数对于调优 AI 回答效果非常重要：

1）model（模型选择）

不同模型有不同的特点和价格：

- text-davinci-003：最强大的 GPT-3 模型，适合复杂任务
- text-curie-001：速度快，成本低，适合简单任务
- text-babbage-001：更快更便宜，适合分类等简单任务
- text-ada-001：最快最便宜，适合关键词提取等任务

选择建议：对于问答场景，建议使用 `text-davinci-003`。

2）max_tokens（最大输出长度）

控制生成文本的最大长度：

- 1 token ≈ 4 个英文字符或 1-2 个中文字符
- 设置过小可能导致回答不完整
- 设置过大会增加成本和响应时间
- 建议设置为 1024，可以满足大部分问答需求

3）temperature（随机性控制）

控制输出的随机性和创造性：

- 取值范围：0-2
- 0：输出最确定，适合事实性问答
- 1：平衡创造性和一致性
- 2：最具创造性，但可能不够连贯

建议：问答场景建议使用 0-0.3，保证回答的准确性。

4）top_p（核心采样）

另一种控制随机性的方法：

- 取值范围：0-1

- 0.1：只考虑概率最高的 10% 的词
- 1：考虑所有可能的词

建议：通常与 temperature 配合使用，或者只使用其中一个。

5）frequency_penalty 和 presence_penalty

控制重复内容的生成：

- frequency_penalty：惩罚高频词，减少重复
- presence_penalty：惩罚已出现的词，增加多样性
- 取值范围：-2.0 到 2.0

建议：问答场景通常设置为 0，避免影响回答质量。

二、OpenAI 集成

请求和响应模型

首先定义强类型的请求和响应模型，直接利用 AI 生成即可。

1、创建请求模型
▼
java
复制代码
```
@Data
public class CreateCompletionRequest {

    /**
     * 模型
     */
    private String model;

    /**
     * 提示词
     */
    private String prompt;

    private Integer max_tokens;

    private Integer temperature;

    private Integer top_p;

    private Integer n;

    private Boolean stream;

    private Integer logprobs;

    private String stop;
}
```

2、创建响应模型
▼
java
复制代码
```
@Data
public class CreateCompletionResponse {
```

```
    private Integer created;
    private Usage usage;
    private String model;
    private String id;
    /**
     * 回答列表
     */
    private List<ChoicesItem> choices;
    private String object;

    @Data
    public static class ChoicesItem {
        private String finishReason;
        private Integer index;
        private String text;
        private Integer logprobs;
    }

    @Data
    public static class Usage {
        private Integer completionTokens;
        private Integer promptTokens;
        private Integer totalTokens;
    }
}
```

OpenAI API 封装

项目中的 OpenAI API 封装实现如下，利用 Hutool 工具库发送请求：

▼

java

复制代码

```
@Service
public class OpenAiApi {

    /**
     * 补全
     *
     * @param request
     * @param openAiApiKey
     * @return
     */
    public CreateCompletionResponse createCompletion(CreateCompletionRequest request, String openAiApiKey) {
        if (StringUtils.isBlank(openAiApiKey)) {
            throw new BusinessException(ErrorCode.PARAMS_ERROR, "未传 openAiApiKey");
        }
        String url = "https://api.openai.com/v1/completions";
        String json = JSONUtil.toJsonStr(request);
        String result = HttpRequest.post(url)
                .header("Authorization", "Bearer " + openAiApiKey)
                .body(json)
                .execute()
                .body();
        return JSONUtil.toBean(result, CreateCompletionResponse.class);
    }
}
```

配置管理

项目中的 OpenAI 配置类实现如下：

▼

java

复制代码

```
@Configuration
@ConfigurationProperties(prefix = "openai")
@Data
public class OpenAiConfig {

    /**
     * 模型
```

```
     */
    private String model = ModelConstant.TEXT_DAVINCI_003;

    /**
     * apiKey
     */
    private String apiKey;
}
```

对应的配置文件：

▼
yaml
复制代码
```
openai:
  model: ${OPENAI_MODEL:text-davinci-003}
  apiKey: ${OPENAI_API_KEY:你的apiKey}
```

错误处理

AI 调用有可能会失败，因此要做好参数校验和错误处理，比如在 OpenAI API 调用前进行参数校验：

▼
java
复制代码
```
public CreateCompletionResponse createCompletion(CreateCompletionRequest request, String openAiApiKey) {
    if (StringUtils.isBlank(openAiApiKey)) {
        throw new BusinessException(ErrorCode.PARAMS_ERROR, "未传 openAiApiKey");
    }
    // ... API 调用逻辑
}
```

最好通过阅读官方文档了解 AI 可能会出现的错误类型，手动捕获和分类处理，比如：

- 网络异常：连接超时、网络不可达等
- 认证错误：API Key 无效或过期
- 限流错误：请求频率超过限制
- 服务器错误：OpenAI 服务端异常

不过其他错误也都会被全局异常处理器统一处理，返回统一的错误响应。

参数调优

在 OpenAiAnswerer 中，我们设置了以下参数，确保了回答的质量和稳定性：

▼
java
复制代码
```
@Override
public String doAnswer(String prompt) {
    CreateCompletionRequest request = new CreateCompletionRequest();
    request.setPrompt(prompt);
    request.setModel(openAiConfig.getModel());
    request.setTemperature(0);        // 设置为0，确保回答的确定性
    request.setMax_tokens(1024);      // 限制最大输出长度
    CreateCompletionResponse response = openAiApi.createCompletion(request, openAiConfig.getApiKey());
    // ... 处理响应
}
```

参数说明：

- temperature = 0：确保回答的确定性和准确性，适合问答场景
- max_tokens = 1024：平衡回答完整性和成本控制
- model：通过配置文件灵活设置，默认使用 text-davinci-003

三、回答者模块

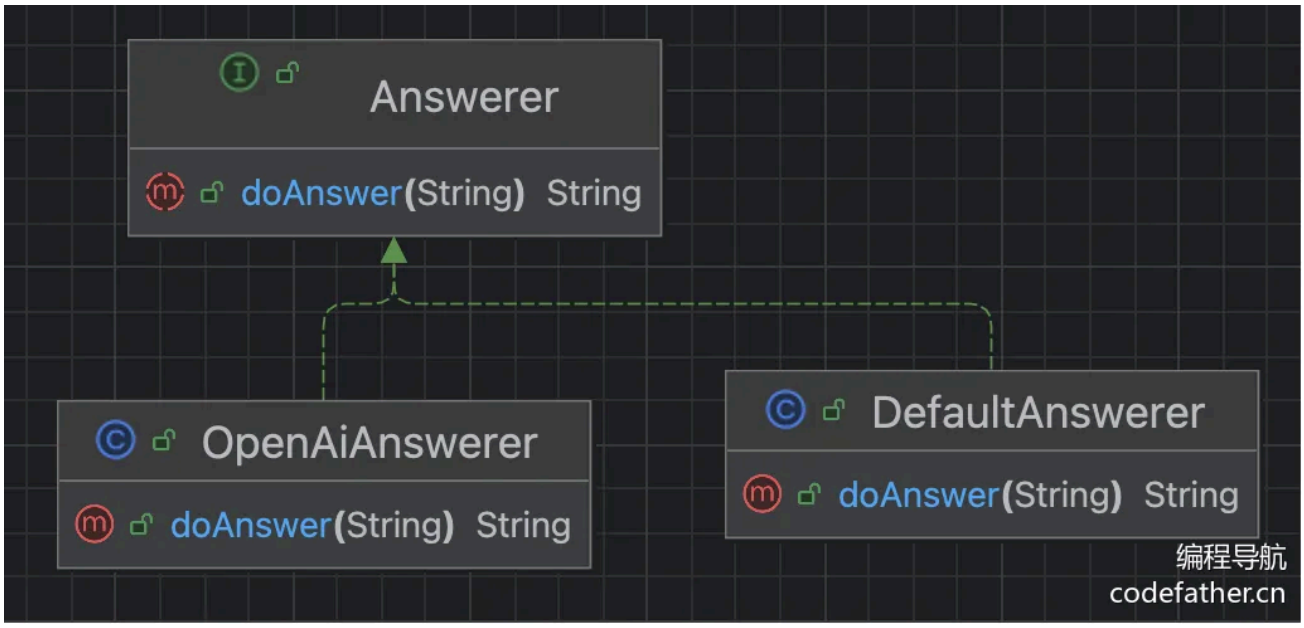
OpenAI 回答者

回答这的实现比较简单，主要是调用 OpenAI 得到回答。代码如下：
▼
java
复制代码
```
@Slf4j
public class OpenAiAnswerer implements Answerer {

    private final OpenAiApi openAiApi = SpringContextUtils.getBean(OpenAiApi.class);

    private final OpenAiConfig openAiConfig = SpringContextUtils.getBean(OpenAiConfig.class);

    @Override
    public String doAnswer(String prompt) {
        CreateCompletionRequest request = new CreateCompletionRequest();
        request.setPrompt(prompt);
        request.setModel(openAiConfig.getModel());
        request.setTemperature(0);
        request.setMax_tokens(1024);
        CreateCompletionResponse response = openAiApi.createCompletion(request, openAiConfig.getApiKey());
        List<CreateCompletionResponse.ChoicesItem> choicesItemList = response.getChoices();
        String answer = choicesItemList.stream()
                .map(CreateCompletionResponse.ChoicesItem::getText)
                .collect(Collectors.joining());
        log.info("OpenAiAnswerer 回答成功 \n 答案：{}", answer);
        return answer;
    }
}
```

默认回答者

项目提供了默认回答者，作为项目扩展的参考和降级实现：
▼
java
复制代码
```
public class DefaultAnswerer implements Answerer {

    @Override
    public String doAnswer(String prompt) {
        return "抱歉，我不理解您的问题：" + prompt;
    }
}
```

现有的回答者结构如图：



回答者工厂

用于根据字符串创建回答者，代码如下：

▼

```
java
复制代码
public class AnswererFactory {

    /**
     * 创建回答者
     *
     * @param answerer
     * @return
     */
    public static Answerer createAnswerer(String answerer) {
        switch (answerer) {
            case "openai":
                return new OpenAiAnswerer();
            default:
                return new DefaultAnswerer();
        }
    }
}
```

四、扩展思路

1）集成更多的 AI 模型，比如文心一言、通义千问、甚至是 Claude 等

2）添加回答的个性化定制，根据用户偏好调整回答风格（成本较高，需要实现用户模块）

3）实现 AI 调用降级机制，如果主模型调用失败，改为调用备选模型。